

京东前端一面面试题，你会几道？

京东前端一面

1. 请手写实现call apply bind
2. 说一说你知道的前端性能优化方案
3. 实现一个插入排序
4. 如何防御 XSS 攻击？
5. 实现一个add方法完成两个大数相加
6. margin 和 padding 的使用场景
7. JavaScript为什么要进行变量提升，它导致了什么问题？
8. 如何获得对象非原型链上的属性？
9. || 和 && 操作符的返回值？
10. TLS/SSL的工作原理
11. 说一说SessionStorage和localStorage还有cookie
12. Node 中的 Event Loop 和浏览器中的有什么区别？ process.nextTick 执行顺序？
13. title与h1的区别、b与strong的区别、i与em的区别？
14. 同样是重定向，307，303，302的区别？
15. 进程与线程的概念
16. 如何解决 1px 问题？
17. 渐进增强和优雅降级之间的区别
18. 如何用webpack来优化前端性能？
19. 什么是margin重叠问题？如何解决？

参考答案：

1. 手写实现call, apply, bind

```
1 // 实现call
2 Function.prototype.myCall = function(context, ...args) {
3   context = context || globalThis; // context 为 null 或 undefined 时，指向全局对象
```

```

4   const fn = Symbol('fn'); // 创建一个唯一的属性
5   context[fn] = this; // this 指向调用 myCall 的函数
6   const result = context[fn](...args); // 执行函数
7   delete context[fn]; // 删除属性
8   return result;
9 };
10
11 // 实现apply
12 Function.prototype.myApply = function(context, args) {
13   context = context || globalThis;
14   const fn = Symbol('fn');
15   context[fn] = this;
16   const result = Array.isArray(args) ? context[fn](...args) : context[fn]();
17   delete context[fn];
18   return result;
19 };
20
21 // 实现bind
22 Function.prototype.myBind = function(context, ...args) {
23   const fn = this;
24   return function(...newArgs) {
25     return fn.apply(context, args.concat(newArgs));
26   };
27 };

```

2. 前端性能优化方案

- 减少HTTP请求：合并文件、使用雪碧图、Inline小资源。
- 使用CDN：将资源分布到多个服务器上，提高加载速度。
- 压缩文件：压缩HTML、CSS、JavaScript文件，移除不必要的空格和注释。
- 缓存策略：使用缓存头部（如 `Cache-Control`）、利用Service Worker进行缓存。
- 懒加载和异步加载：图片懒加载、脚本异步加载（`async` 或 `defer`）。
- 优化图片：使用适当的图片格式（如WebP），压缩图片。
- 减少重绘和重排：避免频繁操作DOM，使用 `requestAnimationFrame`。
- 代码分割：使用Webpack等工具进行代码分割，按需加载模块。
- 预加载资源：使用 `<link rel="preload">`、`<link rel="prefetch">`。

3. 实现一个插入排序

```

1 function insertionSort(arr) {
2   for (let i = 1; i < arr.length; i++) {
3     let key = arr[i];

```

```

4     let j = i - 1;
5     while (j >= 0 && arr[j] > key) {
6         arr[j + 1] = arr[j];
7         j--;
8     }
9     arr[j + 1] = key;
10 }
11 return arr;
12 }

```

4. 防御XSS攻击

- 输入验证：严格验证用户输入，确保数据格式和内容符合预期。
- 输出编码：对用户输入的内容进行HTML编码，防止恶意代码执行。
- 使用安全的API：使用 `innerText` 或 `textContent` 代替 `innerHTML`。
- CSP（内容安全策略）：通过CSP头部设置，限制加载的资源 and 执行的脚本。
- 避免内联脚本和事件处理程序：将JavaScript代码移到外部文件，避免在HTML中内联JavaScript。
- 安全的库和框架：使用安全的库和框架（如React、Angular）来防止XSS。

5. 实现一个add方法完成两个大数相加

```

1 function add(a, b) {
2     let res = '';
3     let carry = 0;
4
5     a = a.split('').reverse();
6     b = b.split('').reverse();
7
8     const maxLength = Math.max(a.length, b.length);
9     for (let i = 0; i < maxLength; i++) {
10         let sum = (parseInt(a[i] || '0') + parseInt(b[i] || '0') + carry);
11         carry = Math.floor(sum / 10);
12         res = (sum % 10) + res;
13     }
14
15     if (carry > 0) {
16         res = carry + res;
17     }
18
19     return res;
20 }

```

6. margin 和 padding 的使用场景

- margin：用于控制元素与外部（其他元素或边界）之间的距离。常用于布局 and 分隔内容。
- padding：用于控制元素内部内容与其边框之间的距离。常用于增加内容的可读性和视觉效果。

7. JavaScript变量提升的原因及问题

变量提升是指JavaScript在执行代码之前会先扫描所有的变量声明和函数声明，并将它们提升到作用域的顶部。导致的问题包括：

- 未初始化的变量：访问提升但未初始化的变量会返回 `undefined`。
- 重复声明：可能导致变量覆盖问题。

```
1 console.log(foo); // undefined
2 var foo = 1;
3
4 console.log(bar); // ReferenceError: bar is not defined
5 let bar = 2;
```

8. 获得对象非原型链上的属性

```
1 const obj = { a: 1, b: 2 };
2 console.log(Object.keys(obj)); // ["a", "b"]
3
4 for (let key in obj) {
5   if (obj.hasOwnProperty(key)) {
6     console.log(key); // "a", "b"
7   }
8 }
```

9. || 和 && 操作符的返回值

- ||（逻辑或）：返回第一个真值，或最后一个假值。
- &&（逻辑与）：返回第一个假值，或最后一个真值。

```
1 console.log(false || 'default'); // 'default'
2 console.log('value' || 'default'); // 'value'
3
4 console.log(true && 'value'); // 'value'
5 console.log(false && 'value'); // false
```

10. TLS/SSL的工作原理

- 握手阶段：
 - 客户端发送支持的加密算法和随机数。
 - 服务器选择加密算法，返回证书和随机数。
 - 客户端验证证书，生成会话密钥，并加密发送给服务器。
 - 服务器解密会话密钥，握手完成。
- 数据传输阶段：
 - 使用会话密钥加密数据传输。

11. SessionStorage、localStorage和cookie

- SessionStorage：临时存储数据，关闭浏览器窗口后数据丢失。
- localStorage：持久存储数据，数据不会随浏览器关闭而丢失。
- cookie：小数据存储（每个最多4KB），随每个HTTP请求发送到服务器。

12. Node中的Event Loop和浏览器中的区别及process.nextTick顺序

- Node：基于libuv库实现，分为多个阶段（timers、I/O callbacks、idle、poll、check、close callbacks）。
- 浏览器：基于HTML5标准，实现了宏任务和微任务队列。

process.nextTick在Node中会在当前阶段的任务执行完后立即执行，优先级高于微任务（Promise）。

13. title与h1、b与strong、i与em的区别

- title：页面标题，显示在浏览器标签上。
- h1：页面内容的一级标题，用于表示内容的层次结构。
- b：表示加粗文本，纯样式。
- strong：表示强调的文本，具有语义。
- i：表示斜体文本，纯样式。
- em：表示强调的文本，具有语义。

14. 307、303、302的区别

- 302：临时重定向，可能改变请求方法。
- 303：临时重定向，强制使用GET方法。
- 307：临时重定向，保持请求方法不变。

15. 进程与线程的概念

- 进程：操作系统分配资源的基本单位，每个进程都有独立的内存空间。

- 线程：进程中的执行单位，线程共享进程的内存空间。

16. 解决1px问题

- 使用媒体查询：针对高DPR设备，设置 `border` 宽度。

```
1 @media only screen and (-webkit-min-device-pixel-ratio: 2) {
2   .element {
3     border-width: 0.5px;
4   }
5 }
```

- 使用伪元素：利用伪元素和transform缩放。

```
1 .element::after {
2   content: "";
3   display: block;
4   width: 100%;
5   height: 1px;
6   transform: scaleY(0.5);
7 }
```

17. 渐进增强和优雅降级的区别

- 渐进增强：从基本功能开始，逐步增加功能，以适应高级浏览器。
- 优雅降级：从高级功能开始，逐步减少功能，以适应低级浏览器。

18. 使用webpack优化前端性能

- 代码分割：使用 `SplitChunksPlugin` 实现代码分割，按需加载。
- Tree Shaking：移除未使用的代码，减小包体积。
- 压缩代码：使用 `TerserPlugin` 压缩JavaScript，使用 `css-minimizer-webpack-plugin` 压缩CSS。
- 异步加载：使用 `dynamic import` 实现异步加载。

19. margin重叠问题及解决方法

margin重叠：相邻块级元素的垂直外边距合并为较大的那个外边距。

解决方法：

- 使用padding代替margin。

- 添加边框或内边距。
- 设置浮动。
- 使用 `overflow: hidden`。

```
1 .parent {  
2   overflow: hidden;  
3 }
```